

M2 - Architecture et Programmation d'accélérateurs Matériels.

(APM 2025-2026)



TD Noté

Prise en main de CUDA
Visualisation d'un objet 3D
représenté par une fonction de
distance

julien.jaeger@cea.fr
thibaut.pepin@cea.fr

Les objectifs de ce TD sont :

- Configuration de grilles et blocs dépendant du problème posé
- Écriture de kernels CUDA
- Déplacement mémoire entre l'accélérateur et le processeur
- Mapping de tableau multidimensionnel sur une unique zone mémoire contigüe

I Visualisation d'objet défini par SDF

1 Description du problème

En visualisation, la méthode la plus utilisée pour représenter un objet consiste à construire la surface de cet objet en utilisant des triangles, puis à utiliser un algorithme pour projeter ces triangles sur l'écran, en respectant la profondeur. Une autre méthode de représentation fonctionne à partir d'une fonction décrivant pour chaque objet la distance entre un point dans l'espace et la surface de cet objet. La distance retournée sera négative si le point est à l'intérieur de la forme.

Vous essaierez ici de visualiser un objet à partir de cette seconde méthode. La fonction de distance étant déjà implémentée dans le template à votre disposition.

Cette visualisation s'effectuera en deux étapes représentées par **différents** kernels :

- Échantillonner un espace en trois dimensions à l'aide de la fonction de distance. Ces échantillons seront répartis uniformément selon une grille comme présenté figure 1. Pour chaque échantillon, nous associerons une couleur qui dépendra de l'orientation du gradient du champ de distance, ou noir si l'échantillon est en dehors de la forme.
- Nous disposons maintenant d'un tableau 3D de couleur. Afin de projeter ces données pour les visualiser sur un support 2D (l'écran), nous imaginons un observateur situé aux coordonnées $(0, +\infty, 0)$ et ne percevant pas la perspective. Pour obtenir l'image final, il suffit alors de parcourir le tableau de couleurs depuis la coordonnée 'y' la plus petite vers la plus grande en conservant uniquement la couleur la plus récente rencontrée, différente du noir, comme présenté figure 2.

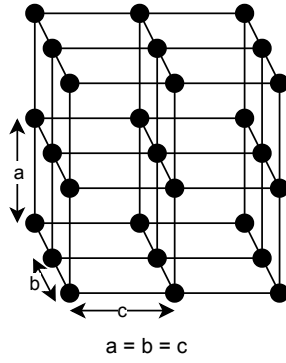


FIGURE 1 – Distribution des échantillons dans un espace 3D

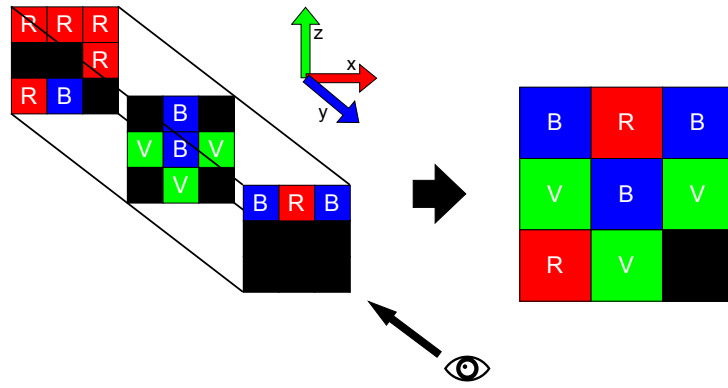


FIGURE 2 – Projection sans perspective d'une tableau 3D vers un tableau 2D

2 Transposition du problème

2.1 Allocation mémoire

Pour stocker les couleurs, vous utiliserez un tableau de taille $1000 \times 1000 \times 1000$. La structure de couleur à utiliser vous est fournie dans le template de code ('Color').

Q.1: Afin de simplifier les transferts mémoire à effectuer, bien que vos couleurs soient organisées en 3D (chacune d'entre elle est liée à une position dans un espace 3D), vous utiliserez un tableau 1D pour les stocker. La figure 3 vous indique l'index dans le tableau à une dimension en fonction de leur position dans l'espace 3D. Écrivez une fonction renvoyant l'index à utiliser dans le tableau 1D à partir des index (x, y, z) que vous utiliseriez si vous utilisiez un tableau 3D (type Color *** : tableau de tableau de tableau).

Pensez à remplir l'accès au tableau de couleur dans la fonction 'PPMSave'. L'accès souhaité se situe à l'index $(x, slice, z)$.

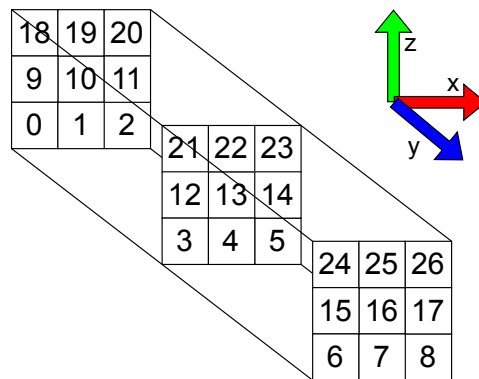


FIGURE 3 – Index du tableau 1D dans le tableau 3D.

Q.2: Allouez un tableau 1D de la bonne taille sur l'host, puis, en utilisant les fonctions du SDK CUDA, sur le device.

2.2 Échantillonnage

L'objet à visualiser est centré à la coordonnée $(0, 0, 0)$ et on souhaite observer la partie de l'espace allant de $(-1, -1, -1)$ à $(1, 1, 1)$. Chaque thread s'occupera d'évaluer la fonction de distance ('GetDistance') à une unique coordonnée.

Q.3: Proposez une structure de grille et de bloc pour les threads CUDA afin de repartir l'espace à échantillonner parmi les threads. Justifier votre choix.

Q.4: Les couleurs obtenues à la fin de l'exécution du kernel gérant l'échantillonnage doivent être laissées sur le device pour la prochaine étape. Cependant afin de pouvoir évaluer si votre kernel est correct, effectuez une copie mémoire vers l'host et sauvegardez la tranche de votre tableau correspondant aux échantillons situés sur le plan XZ de l'espace intersectant $Y=0$. Utilisez pour cela la fonction 'PPMSave' qui vous est fournie. Le paramètre 'slice' de cette fonction correspond à l'index 'y' qui reste constant pendant la lecture du tableau de couleurs.

Attention, l'index $y=0$ dans le tableau 3D ne correspond pas aux valeurs échantillonnées sur le plan XZ de l'espace intersectant $Y=0$. Votre tableau 3D de côté 1000 contient des échantillons effectués dans un cube dans l'espace 3D de côté 2 centré en $(0, 0, 0)$. Vous devriez obtenir la même image que la figure 4.

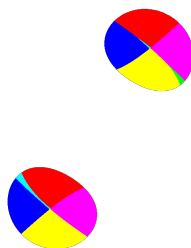


FIGURE 4 – Tranche de l'objet à visualiser, plan XZ, $Y=0$.

Q.5: Passons maintenant à l'écriture du kernel. Chaque thread devra déterminer une unique position dans l'espace où il devra échantillonner la fonction de distance. Vous utiliserez pour cela les bornes de l'espace à observer, la taille du tableau à remplir ainsi que les index et taille des grid/bloc/thread CUDA. Pensez à centrer l'espace à observer sur $(0, 0, 0)$ et n'oubliez pas l'organisation des données présentée figure 3. Une fois la position déterminée, utiliser la fonction 'GetColor' pour obtenir la couleur et remplissez votre tableau de couleur au bon index. Cette fonction déterminera pour vous quelle couleur à utiliser en fonction de l'orientation du gradient du champ de distance.

2.3 Projection

Chaque thread devra s'occuper d'une 'colonne' de valeurs dans le tableau 3D de couleurs en fixant x et z et en faisant varier y . De la même manière que la partie précédente, une fois le kernel de projection exécuté, copiez le tableau de

couleur vers l'hôte et utilisez la fonction 'PPMSave' pour obtenir le résultat final et révéler la forme que l'on cherche à visualiser.

Q.6: Commencez par proposer une structure de grille et de bloc pour les threads CUDA afin de répartir les colonnes de données parmi les threads pour effectuer la projection. Justifier votre choix.

Q.7: Passons à l'écriture du kernel. Chaque thread devra déterminer les index (x, z) de la colonne du tableau de couleurs dont il s'occupera. Puis, chaque thread itérera depuis $y = 0$ vers $y = 1000$, et enregistra à la base de la colonne ($y=0$) la dernière couleur rencontrée qui soit différente de noir ($r=0, g=0, b=0$).

II Rendu

Vous enverrez un unique fichier contenant le code produit à l'adresse suivante : `thibaut.pepin@cea.fr`

Les réponses aux questions devront être en commentaire dans le code.

Vous aurez jusqu'au 13 novembre à 23h59 Pour rendre votre travail. 1 point sera soustrait par tranche de 10 minutes de retard.